



UNIDADE IV

Desenvolvimento Web com .NET

Prof. Dr. Tarcísio Peres

Observabilidade: entender a aplicação em execução

- Definição: permite inferir o estado interno do sistema a partir dos sinais produzidos durante o uso.
- Objetivo: investigar falhas, lentidão, erros intermitentes e comportamentos inesperados.
- Valor: substitui suposições por evidências sobre o que ocorreu, onde e em qual contexto.
- Sinais principais: logs, métricas, tracing e health checks oferecem perspectivas complementares.
- Critério: ser observável não é registrar tudo, mas produzir sinais úteis, contextualizados e seguros.

Cada sinal responde a uma pergunta diferente

- Logs: o que aconteceu em um evento específico e qual era o contexto da operação?
- Métricas: com que frequência o comportamento ocorre e como ele evolui ao longo do tempo?
- Tracing: por quais etapas a operação passou e onde consumiu tempo ou encontrou uma falha?
- Health checks: o processo está vivo e as dependências essenciais estão prontas para operar?
- Leitura conjunta: os quatro sinais encurtam o caminho entre sintoma, causa e ação corretiva.

Logs registram eventos com nível e contexto

- Registro de eventos: narra acontecimentos relevantes, como início, conclusão, rejeição e falha de operações.
- Information: descreve uma operação esperada, como a renderização de uma listagem.
- Warning: sinaliza uma condição suspeita ou que merece atenção, como um login inválido.
- Error: registra uma falha que impediu a conclusão da operação.
- Contexto: inclua operação, página, identificador e resultado quando esses dados ajudam no diagnóstico.
- Equilíbrio: logs demais geram ruído; logs insuficientes tornam o problema invisível.

Logs estruturados facilitam busca sem expor segredos

- Estrutura: use propriedades nomeadas em vez de concatenar valores em uma única mensagem.
- Exemplo: “Listagem solicitada. Página: {Page}; tamanho: {PageSize}”.
- Benefício: campos preservados podem ser pesquisados, filtrados e analisados por ferramentas de coleta.
- Autenticação: registre o resultado da tentativa e apenas o contexto operacional necessário.
- Proteção: nunca grave senha, cookie, token, segredo ou dado sensível nos logs.
- Regra prática: registre o suficiente para diagnosticar, sem transformar o log em cópia dos dados do usuário.

Métricas revelam volume, frequência e tendência

- Função: agregam medições do comportamento da aplicação ao longo do tempo.
- Contadores: podem registrar listagens, detalhes consultados, logins bem-sucedidos e logins inválidos.
- Exemplos: `pet_list_requests`, `pet_details_requests`, `successful_logins` e `invalid_logins`.
- Diferença central: um log explica um evento; uma métrica evidencia um padrão.
- Tendências: mostram picos de uso, degradação progressiva, gargalos recorrentes e efeitos de mudanças.
- Uso didático: `System.Diagnostics.Metrics` permite criar contadores nativos no .NET.

Interatividade

Qual alternativa descreve melhor a função dos Health Checks?

- a) Criar usuários automaticamente.
- b) Verificar objetivamente se a aplicação ou suas dependências estão saudáveis.
- c) Substituir logs e métricas.
- d) Impedir todas as exceções da aplicação.
- e) Gerar páginas Razor.

Resposta

Qual alternativa descreve melhor a função dos Health Checks?

- a) Criar usuários automaticamente.
- b) Verificar objetivamente se a aplicação ou suas dependências estão saudáveis.
- c) Substituir logs e métricas.
- d) Impedir todas as exceções da aplicação.
- e) Gerar páginas Razor.

Tracing localiza tempo e falhas no percurso da operação

- Origem: ActivitySource centraliza a criação e a identificação das atividades observadas.
- Atividade: representa uma operação relevante, como listar pets ou executar um fluxo de autenticação.
- Percurso: acompanha endpoint, filtros, regras, consulta ao banco, chamadas externas e resposta.
- Diagnóstico: revela em qual etapa o tempo foi gasto ou a falha ocorreu.
- Correlação: conecta etapas da mesma operação, mesmo quando o problema não está em um único ponto.
- Exemplo: uma atividade “Listar pets” pode envolver consulta, paginação e geração do resultado.

Health checks distinguem vida de prontidão

- Vida — /health/live: confirma que o processo está ativo e consegue responder.
- Prontidão — /health/ready: informa se a aplicação está em condição de atender corretamente.
- Dependências: a prontidão pode verificar banco de dados e outros componentes essenciais.
- Interpretação: o processo pode estar vivo e, ao mesmo tempo, não estar pronto por causa de uma dependência.
- Implementação: uma consulta mínima pelo DbContext pode confirmar o acesso ao SQLite.
- Uso operacional: endpoints distintos permitem identificar rapidamente o tipo de indisponibilidade.

Resiliência contém falhas e preserva o serviço

- Definição: capacidade de manter comportamento adequado quando algo falha ou demora além do esperado.
- Meta: impedir que uma falha localizada comprometa todo o sistema.
- Mecanismos: incluem timeout, retry, circuit breaker, fallback, isolamento e limites de carga.
- Falhas esperadas: rede, serviços externos e banco podem ficar lentos ou indisponíveis temporariamente.
- Experiência: respostas controladas evitam telas quebradas, mensagens técnicas e estados inconsistentes.
- Relação: observabilidade mostra como o sistema falha; resiliência limita o impacto e apoia a recuperação.

Retry, circuit breaker e fallback têm papéis distintos

- Retry: repete falhas transitórias com número máximo de tentativas, intervalo e critérios definidos.
- Aplicação segura: é mais adequado a leituras e operações idempotentes; gravações exigem cautela.
- Circuit breaker: abre o circuito após falhas repetidas e evita chamadas a uma dependência instável.
- Recuperação: depois de um intervalo, permite nova tentativa para verificar se a dependência voltou.
- Fallback: entrega uma resposta alternativa, clara e segura quando a função principal não está disponível.
- Exemplo: a busca de uma dica pode tentar até três vezes e, ao falhar, retornar uma orientação padrão.

Rate limiting e output caching protegem capacidade

- Rate limiting: limita requisições aceitas em determinado intervalo para reduzir excesso e abuso.
- Políticas: leituras públicas podem ter limites maiores; login e cadastro exigem limites mais restritos.
- Excesso: quando o limite é ultrapassado, a API pode retornar 429 Too Many Requests.
- Output caching: reutiliza por pouco tempo uma resposta pública e independente do usuário.
- Benefício: reduz processamento e consultas repetidas em listagens, detalhes e conteúdos públicos.
- Não aplicar: páginas personalizadas, autenticadas ou dependentes de cookie, claims e sessão ficam fora do cache.
- Efeito conjunto: o limite controla a demanda e o cache reduz o custo das respostas repetidas.

Interatividade

Qual é a finalidade principal do circuit breaker?

- a) Repetir indefinidamente uma requisição com falha.
- b) Bloquear todos os usuários anônimos.
- c) Evitar chamadas repetidas a uma dependência que está falhando.
- d) Armazenar respostas HTML em cache.
- e) Validar campos de formulário.

Resposta

Qual é a finalidade principal do circuit breaker?

- a) Repetir indefinidamente uma requisição com falha.
- b) Bloquear todos os usuários anônimos.
- c) Evitar chamadas repetidas a uma dependência que está falhando.
- d) Armazenar respostas HTML em cache.
- e) Validar campos de formulário.

Build: do código ao artefato executável

- Build transforma código, dependências, configurações seguras e recursos em um artefato executável ou publicável.
- No .NET, compila o código C#, verifica referências, prepara arquivos estáticos e gera o publish da aplicação.
- No React, converte componentes, módulos, estilos, imagens e scripts em arquivos otimizados para o navegador.
- Em full-stack, o processo precisa reunir back-end e front-end e confirmar que as partes funcionam de modo coerente.
- Saída: um pacote identificável, pronto para validação e para implantação em um ambiente de destino.

Implantação coloca a aplicação em uso real

- Implantar é publicar a aplicação em um ambiente acessível aos usuários; não é apenas gerar o build.
- O ambiente precisa fornecer banco de dados, URLs de API, arquivos estáticos, certificados, permissões e políticas de segurança.
- Configuração e dependências devem ser explícitas: algo que funciona localmente pode falhar em produção.
- Observabilidade com logs, métricas, rastreamentos e Health Checks ajuda a confirmar o comportamento da nova versão.
- Full-stack exige compatibilidade entre API, interface, banco, autenticação e comunicação entre camadas.

CI/CD torna a entrega repetível e verificável

- CI integra alterações frequentes e executa verificações controladas sobre o código.
- Fluxo típico: obter o código, restaurar dependências, compilar, testar, empacotar e preparar a entrega.
- CD mantém a versão pronta para entrega ou automatiza a implantação, conforme o grau de automação adotado.
- Pipeline versionado reduz arquivos esquecidos, diferenças entre máquinas e mudanças manuais sem registro.
- Resultado: erros detectados mais cedo e relação rastreável entre código validado e versão publicada.

Cada ambiente usa configuração própria

- Princípio: a mesma aplicação muda valores externos sem alterar classes nem recompilar o código.
- Development: diagnóstico detalhado, execução local, URLs locais e dados controlados.
- Staging: validação em condições próximas às de produção, com conexões e serviços próprios.
- Production: menor exposição de detalhes, configuração segura e foco em disponibilidade.
- ASP.NET Core combina appsettings.json, arquivos por ambiente e variáveis de ambiente em uma configuração unificada.

Secrets ficam fora do código e da imagem

- São secrets: senhas, tokens, chaves de API e strings de conexão reais.
- Não são secrets: nome público da aplicação, mensagem visual segura e parâmetros operacionais sem credenciais.
- Nunca armazenar em código, appsettings versionado, Dockerfile, imagem do container ou capturas de tela.
- Na automação, o pipeline pode referenciar o nome de um secret do ambiente, mas nunca registrar seu valor.
- Separação essencial: o artefato descreve a aplicação; o ambiente fornece os valores sensíveis.

Interatividade

Qual alternativa descreve corretamente o papel de um secret?

- a) É qualquer texto exibido ao usuário.
- b) É uma configuração sensível, como senha, token ou chave de API.
- c) É sempre um arquivo CSS.
- d) É o artefato final do build.
- e) É uma rota pública da aplicação.

Resposta

Qual alternativa descreve corretamente o papel de um secret?

- a) É qualquer texto exibido ao usuário.
- b) É uma configuração sensível, como senha, token ou chave de API.**
- c) É sempre um arquivo CSS.
- d) É o artefato final do build.
- e) É uma rota pública da aplicação.

User Secrets no local; variáveis no servidor

- User Secrets guardam valores sensíveis no perfil local de desenvolvimento, fora do repositório e associados ao projeto.
- Variáveis de ambiente fornecem configuração externa em servidores e containers e podem sobrescrever appsettings.
- Hierarquia: em ASP.NET Core, dois underlines representam níveis de uma seção de configuração.
- Exemplo: PetMatch:AdoptionTips:ApiKey torna-se PetMatch__AdoptionTips__ApiKey.
- Benefício: a mesma imagem executa em vários ambientes sem incorporar credenciais ao pacote.

Opções tipadas organizam a configuração

- Classes C# representam seções como `PetMatch:Application` e `PetMatch:AdoptionTips`.
- Binding conecta a seção à classe com `Configure<T>` e `GetSection` no `Program.cs`.
- Consumo ocorre por `IOptions<T>` ou `IOptionsMonitor<T>` via injeção de dependência.
- Organização: controllers e serviços deixam de ler arquivos de configuração manualmente.
- Testabilidade melhora porque valores e dependências ficam explícitos; o secret pode continuar vindo de fonte externa.

Containers tornam a execução reproduzível

- Dockerfile descreve como restaurar, publicar e executar a aplicação em uma imagem.
- Build em estágios: a imagem de SDK compila e publica; a imagem de runtime executa somente o resultado.
- Imagem contém aplicação e runtime, mas não senha, token, chave de API ou conexão real.
- Configuração externa entra por variáveis de ambiente ou por mecanismo seguro do ambiente de execução.
- .dockerignore exclui bin, obj, .vs, node_modules e outros arquivos desnecessários do contexto de build.

Deploy integrado simplifica a operação

- Modelo: o ASP.NET Core publica a API e serve os arquivos estáticos gerados pelo React.
- Integração: HTML, CSS, JavaScript e assets entram no mesmo artefato, por exemplo: em `wwwroot/app`.
- Vantagens: um host, um domínio, configuração central e menor exposição a problemas de CORS.
- Rotas devem separar a SPA, a API e os Health Checks para evitar que o fallback capture endpoints do servidor.
- Limite: front-end e back-end compartilham o ciclo de publicação; uma mudança pode exigir o pacote completo.

Deploy separado dá autonomia às partes

- Modelo: API e front-end usam pipelines, ambientes e ciclos de publicação próprios.
- Hospedagem: a API pode ser aplicação web ou container; o React pode ser site estático ou conteúdo em CDN.
- Vantagens: escala, cache, publicação e rollback independentes para cada componente.
- Cuidados: URLs da API, CORS, cookies, autenticação e variáveis do front-end precisam ser coordenados.
- Governança: equipes devem manter compatibilidade entre versões e sincronizar contratos entre os pipelines.

Interatividade

Qual é a principal diferença entre deploy integrado e deploy separado?

- a) No integrado, não existe back-end.
- b) No separado, não existe front-end.
- c) No integrado, API e arquivos estáticos podem sair no mesmo artefato; no separado, API e front-end têm entregas independentes.
- d) No integrado, não há configuração por ambiente.
- e) No separado, não é possível usar CI/CD.

Resposta

Qual é a principal diferença entre deploy integrado e deploy separado?

- a) No integrado, não existe back-end.
- b) No separado, não existe front-end.
- c) No integrado, API e arquivos estáticos podem sair no mesmo artefato; no separado, API e front-end têm entregas independentes.
- d) No integrado, não há configuração por ambiente.
- e) No separado, não é possível usar CI/CD.

ATÉ A PRÓXIMA!